Chapitre 9 :
Stratégies algorithmiques

Sources :
Vazinami, Approximation algorithms
Mohavani & Raghavan, Randomized algorithms

Table des matières

N.1 Introduction : *« Résoudre » un problème NP-Complet* 1

I Alorithmes d'appproximations à facteur constant 2

I.1 Définition 2

I.2 Exemples 3

II Algorithmes probabilistes 7

II.1 Principes et intérêts 7

II.2 Les algorithmes Las Vegas 8

II.3 Algorithmes de Monte Carlo 10

II.4 Liens entre algorithmes Las Vegas et Monte Carlo 11

II.4.a Las Vegas vers Monte Carlo 12

II.4.b De Monte Carlo à Las Vegas 12

II.5 Exemple : algorithme de Freivalds 12

N.1 Introduction : *« Résoudre » un problème NP-Complet*

Pour trouver une solution d'un problème (en particulier d'optimisation) NP-Complet, on a plusieurs idées élémentaires :

1. L'exploration exhaustive (du bruteforce) → possible sur de petites instances
   cf. « branch-and-bound » dans la suite
2. Concevoir un algorithme ad-hoc sur une instance particulière qui nous intéresse
3. Si c'est possible (si le contexte s'y prête, fout pas ça dans un avion connard), on peut se contenter de calculer une solution approchée au problème (algorithmes d'approximation)
4. Si c'est possible, on peut concevoir un algorithme qui
   → Donne la bonne réponse presque tout le temps (et rapidement)
   → Donne la bonne réponse tout le temps en étant presque tout le temps rapide
   Il est ici question d'algorithmes probabilistes

I Alorithmes d'appproximations à facteur constant

I.1 Définition

On se place dans le cadre des problèmes d'optimisation.

Définition

Soit $\mathcal{P}$ un problème d'optimisation
$\mathcal{A}$ un algorithme qui pour tout instance x de $\mathcal{P}$, fourni une solution $\mathcal{A}(x)$ (pas forcément optimale)
$\alpha \in \mathbb{R}_+^*$

On dit que $\mathcal{A}$ est un **algorithme d'approximation à facteur constant** de $\mathcal{P}$ si :

- $\mathcal{P}$ est un problème de minimisation, $\alpha > 1$ et pour toute instance x de $\mathcal{P}$, $C(x, \mathcal{A}(x)) \leq \alpha C^*(x)$ où $C^*(x)$ est le coût d'une solution optimale sur x
- $\mathcal{P}$ est une problème de maximisation, $\alpha < 1$ et pour toute instance x de $\mathcal{P}$, $\alpha C^*(x) \leq C(x, \mathcal{A}(x))$

On dit que a est le **facteur d'approximation** de l'algorithme. On aimerait que α soit le plus proche de 1 possible

Remarque

Pour un problème de minimisation, $\alpha C^*(x) \geq C(x, \mathcal{A}(x)) \geq C^*(x)$
Pour un problème de maximisation, $\alpha C^*(x) \leq C(x, \mathcal{A}(x)) \leq C^*(x)$

Avertissement

Il existe une autre façon de définit un algorithme d'approximation de facteur d'approximation $\alpha \geq 1$:
$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \alpha$

Problème : ce n'est pas cohérent avec la première définition dans le cadre des problèmes de maximisation. Avec le deuxième définition, $\frac{C^*}{C} \leq \alpha$, donc $\frac{C^*}{\alpha} \leq C$

Remarque

Dans le cadre des problèmes NP-Complet, on troque des algorithmes exponentiels exacts contre des algorithmes approchés polynomiaux

⚠ Avertissement

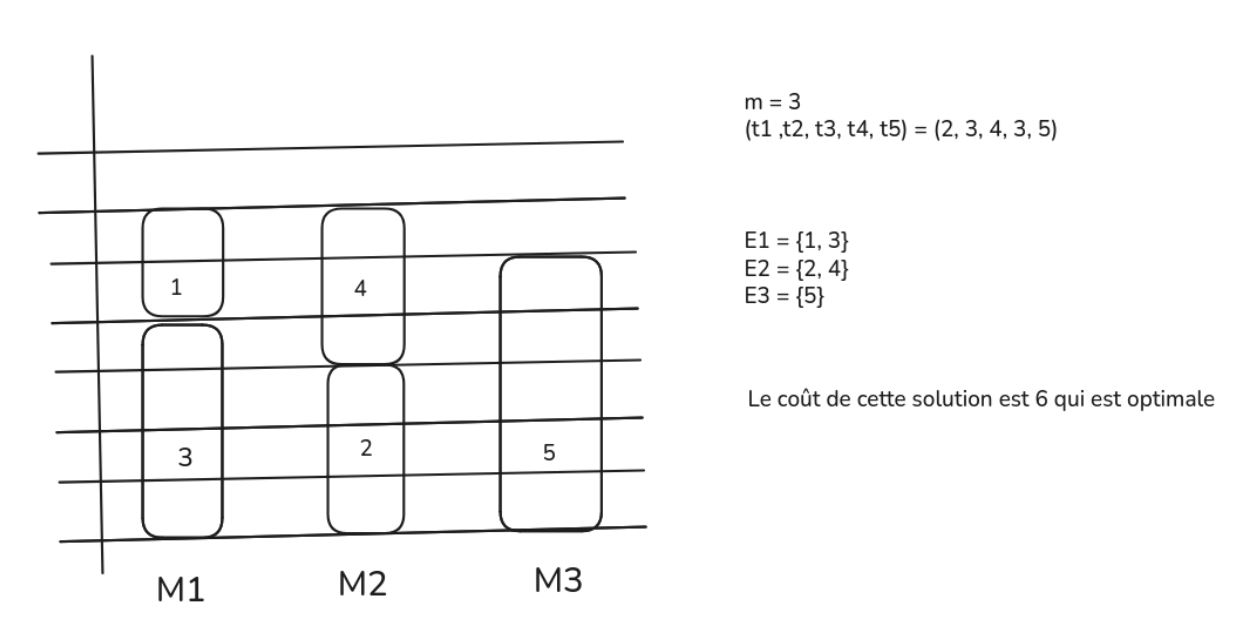
Si $\mathcal{A} \leq \mathcal{B}$ et $\mathcal{A}', \mathcal{B}'$ sont des problèmes d'optimisation associés, connaître un algorithme d'approximation pour $\mathcal{B}$ n'en fournit pas nécessairement un pour $\mathcal{A}$

I.2 Exemples

Exemple 1: On considère le problème ORDONNANCEMENT

$$\begin{cases} \underline{\text{Entrée :}}\ n \in \mathbb{N} \ \text{ tâches de durées } (t_1, ..., t_n) \in \mathbb{N}^n, \ \ m \in \mathbb{N} \ \text{ mâchines} \\ \underline{\text{Solution :}}\ \text{une partition } \{E_1, ..., E_m\} \text{ de } [\![1, n]\!] \\ \underline{\text{Optimisation :}}\ \text{minimiser } \max\limits_{i \in [\![1,m]\!]} \sum\limits_{j \in E_i} t_j \end{cases}$$

La version décisionnelle est NP_complète



Un premier aglorithme est un algorithme glouton où on prend les tâches dans un ordre quelconque, puis on assigne la tâche i à la machine qui termine le calcul le plus tôt au moment où i est considéré

Proposition 1: L'algorithme donné est une 2-approximation pour ORDONNANCEMENT

Démonstation: Soit $m, t_1, ..., t_n$ une instance d'ORDONNANCEMENT. Notons T le temps de traitement pour la solution fournie par l'algorithme sur cette instance, et T^* le temps de traitement optimal sur cette instance.

Notons i l'indice de la machine telle que $\mathrm{T} = \mathrm{T}_i$ et j le numéro de la dernière tâche assignée à la machine i.

$\mathrm{T}_i - t_j$ est plus petit que tous les temps de traitement de toutes les machines au moment où j est considéré

Ainsi, $\mathrm{T}_i - t_j \leq \mathrm{T}^*$, car la machine est la moins chargé, donc toutes les machines ont déjà un temps de traiement supérieur à $\mathrm{T}_i - t_j$ et dans une solution optimale, cela signifie que toutes les machines ont aussi un temps de traitement supérieur à $\mathrm{T}_i - t_j$

Ainsi, $\mathrm{T} = \mathrm{T}_i \leq \mathrm{T}^* + t_j \leq \mathrm{T}^* + \mathrm{T}^* = 2\mathrm{T}^*$

Donc, $\mathrm{T} \leq 2\mathrm{T}^*$ et cet algorithme est une 2-approximation pour ORDONNANCEMENT.

Proposition 2: Le facteur d'approximation de l'algorithme précédent est exactement 2.

Point méthode : Pour montrer qu'un algorithme d'approximation a un facteur d'approximation qui est exactement α

- Trouver une intance telle que $C = \alpha C^*$
- Trouver une suite d'instance telle que le ration entre C et C^* tende vers α

Démonstation: Soit $m \in \mathbb{N}$. Considérons l'instance suivante pour ORDONNANCEMENT : m machines et $m(m-1)$ tâches de durée 1 + une tâche de durée m

Notons T_m le temps de traitement pour la solution fournie par l'algorithme sur cette instance et T_m^* son temps de traitement optimal.

On peut donner la tâche de taille m à la première machine, et m tâches de taille 1 à chaque machine restante. Il y a donc une durée de traîtement de m pour chaque machine, donc $T_m^* = m$

Or, avec l'algorithme, $T_m = 2m - 1$, car on va remplir toutes les machines avec $m - 1$ tâches chacune et mettre la tâche à m au début de la première

Si β est le facteur d'approximation de l'algorithme, on doit avoir
$\forall m \in \mathbb{N}, T_m \leq \beta T_m^*$, donc $\beta \geq \frac{T_m}{T_m^*} = \frac{2m-1}{m}$.
En passant à la limite, on constate que le facteur d'approximation de l'algorithme est supérieur à 2. On a montré qu'il était inférieur à 2, donc il vaut exactement 2.

On trouve ainsi un deuxième algorithme qui se comporte comme le première mais en considérant les tâches par ordre décroissant de durée

Proposition 3: Ce deuxième algorithme est un algorithme d'approximation pour ORDONNANCEMENT de facteur inférieur à $\frac{3}{2}$

Démonstation: On note T le temps de l'algorithme et T^* le temps optimal, $t_1 \geq t_2 \geq ... \geq t_n$

Notons i la machine telle que $T_i = T$ et j la dernière tâche mise dedans.
$T_i - t_j \leq T^*$ par même raisonnement que pour le premier algorithme
Donc $T \leq T^* + t_j$

Or, si $n < m$, l'algorithme donne déjà la solution optimal et $T = T^*$
sinon, si la dernière tâche mise sur i est l'une parmi $1, ..., m$ alors c'est la seule et $T = T^*$
sinon, si la dernière tâche mise sur i est la j-ième avec $j \geq m+1$, alors $T \leq \frac{3}{2}T^*$

Ainsi, pour toute instance, $T \leq \frac{3}{2}T^*$

Exemple 2: Problème du sac à dos KP

$$\begin{cases} \underline{\text{Entrée :}}\ (p_1, ..., p_n) \in \mathbb{N} \text{ des poids} \\ \qquad\qquad (v_n, ..., v_n) \in \mathbb{N} \text{ des valeurs} \\ \qquad\qquad P \in \mathbb{N} \\ \underline{\text{Solution :}}\ I \subset [\![1, n]\!] \text{ tq } \sum_{i \in I} p_i \leq P \\ \underline{\text{Optimisation :}}\ \text{Maximiser} \sum_{i \in I} v_i \end{cases}$$

La variante décisionnelle de ce problème est NP-complet

Un algorithme proposé est celui de considérer les objets par valeurs décroissant et de remplir le sac dans cet ordre tant que ça rentre

Proposition 4: Cet algorithme n'est pas un algorithme d'approximation à facteur constant pour KP

Démonstation: Pour tout $m \in \mathbb{N}$, on considère l'instance de KP suivante :
$P = m$, les objets sont un (m, m) et $m-1$ fois $(m-1, 1)$

La valeur obtenue pour la solution fournie par l'algorithme est $V_m = m$ et la valeur optimale est $V_m^* = m(m-1)$.

Ainsi, si l'algorithme était une α pour KP avec $\alpha > 0$,
on aurait $\forall m \in \mathbb{N}, \alpha V_m^* \leq V_m$, donc $\alpha \leq \frac{V_m}{V_m^*} = \frac{m}{m(m-1)} \underset{m \to +\infty}{\longrightarrow} 0$,
ce qui contredit $\alpha > 0$

Ainsi, il n'existe pas de tel facteur α

Un deuxième algorithme est de trier les objets par ordre décroissant de $\frac{v_i}{p_i}$, et remplir tant que possible

Proposition 5: Cet algorithme n'est toujours pas un algorithme d'approximation pour KP

Démonstation: Pour tout $m \in \mathbb{N}$, on considère l'instance
$P = m$, les objets sont $(m, m), (2, 1)$

$V_m = 2, V_m^* = m$

Proposition 6: Voici une 2-approximation pour KP :

0. Supprimer tous les objets ayant un volume plus grand que celui du sac
1. Trier les objets par valeur décroissante puis les placer dans le sac dans cet ordre s'ils rentrent
   $\rightarrow$ Donne une solution pour laquelle la valeur est V_1
2. Trier les objets par ratio $\frac{\text{valeur}}{\text{poids}}$ décroissant puis les placer dans le sac tant qu'ils rentrent.
   $\rightarrow$ Ceci donne une solution de valeur V_2
3. Choisir la meilleure solution parmis les deux

Démonstation: Soit $(p_1, ..., p_m, v_1, ..., v_m, P)$ une instance de KP

Notons V la valeur du sac obtenu sur cette instance avec cet algorithme et V^* la valeur optimale du sac sur cet instance.

- Si aucun objet ne rentre dans le sac seul, $V = 0 = V^*$, donc $V \geq \frac{1}{2}V^*$
- Sinon, notons $I \subset [\![1, n]\!]$ les indices des objets dans le sac selon 2.
  - Si $I = [\![1, m]\!]$, alors $V = V^*$ donc $V \geq \frac{1}{2}V^*$
  - Sinon, notons j l'indice du premier objet qui ne rentre pas selon 2.
    On a $v_j \leq V_1$ car seul il rentre dans le sac donc 1. a choisi un objet de valeur au moins v_j.

    Si on pouvait prendre des fractions d'objet, on aurait :

$$\begin{aligned} V^* &\leq \left(\sum_{i,\ \text{objets qui rentrent}} v_i \right) + v_j \times (\text{fraction de l'objet } j \text{ pour completer}) \\ &\leq V_2 + v_j \\ &\leq V_2 + V_1 \\ &\leq 2\max(V_1, V_2) = 2V \end{aligned}$$

    Ainsi, $V \geq \frac{1}{2}V^*$ donc l'algorithme est une 2-approximation

II Algorithmes probabilistes

II.1 Principes et intérêts

Définition

Un **algorithme probabiliste** est un enchaînement d'opérations élémentaires, comme pour un algorithme déterministe, à ceci près que dans les opérations élémentaires on rajoute une opération de choix probabiliste consistant à générer un bit.

Ainsi, si la suite de bits aléatoires est deux fois la même, on obtiendra sur la même instance le même résultat. Sinon, une même instance peut engendrer des résultats ou temps de calcul différent

Avertissement

Un algorithme non-déterministe et algorithme probabiliste ne veulent pas dire la même chose

Un algorithme déterministe ne fais pas de choix, un algorithme probabiliste fait des choix probabiliste et un algorithme non déterministe fait un choix au sens où on « devine » l'instruction suivante correspondante

Exemple 3: On considère un tableau de boolénes qui contient des vrai et des faux uniformément, on cherche l'indice d'une case qui contient vrai.

- Un algorithme déterministe : parcourir le tableau jusqu'à trouver une case qui contient V

La complexité pire cas est en O(n) avec n la taille du tableau

- Un algorithme probabiliste :

```
Pour k = 1 à 100
  tirer uniformément aléatoirement aléatoirement i dans [|0, n-1|]
      Si T[i] = V, renvoyer i
renvoyer -1
```

Cet algorithme peut répondre incorrectement mais ça n'arrive qu'une fois sur 2^{100}

Définition

Deux types d'algorithme probabiliste au programme

- Les algorithmes **La Vegas** : toujours correct, probablement rapide
- Les algorithmes **Monte Carlo** : toujours rapide, probablement correct

II.2 Les algorithmes Las Vegas

Définition

Un **algorithme de type Las Vegas** est un algorithme probabiliste dont le résultat est toujours correct mais dont le temps de calcul est une variable aléatoire

Remarque

La terminaison d'un tel algorithme n'est pas garantie mais on impose qu'elle soit presque sûre

Dans le cadre de la MPI, elle sera même sûre

L'objectif ici est de déterminer leur complexité espérée, on fixe une instance de taille n et on estime le temps de calcul en moyennant sur les choix aléatoires.

Exemple 4: L'algorithme de tri rapide :

```
Entrée : Un ensemble S de n valeurs comparables
Sortie : Les éléments de S triés dans l'ordre croissant

Si n ⩽ 1, renvoyer S
Sinon, on choisit un pivot x dans S
    Comparer chaque élément de S\{x} à x pour construire :
      S_sup = {s dans S | s ⩾ x}
      S_inf = {s dans S | s < x}
    Appeler récursivement le tri rapide sur S_sup et S_inf
    Renvoyer (S_inf trié) union {x} union (S_sup trié)
```

Proposition 7: La complexité pire cas d'un tri rapide est en $O(n^2)$ où n est la taille de l'ensemble à trier.

Démonstation: On note $C(n)$ le nombre d'opérations effectués sur un tableau de taille n, l'algorithme donne que $C(n) = \max\limits_{p+q=n-1}(C(p) + C(q)) + O(n)$

Il existe des instances pour laquelle la complexité est quadratique, car si on a $p = n-1$, $q = 0$, à chaque étape :
$C(n) = C(n-1) + O(n)$
Ainsi $C(n) = \sum\limits_{i=0}^{n} i = O(n^2)$

Pour obtenir la majoreation, on peut pracéder par récurrence, c'est à dire montrer par récurrence que $C(n) \leq kn^2$ avec k fixé indépendant de n

Proposition 8: La complexité pire cas du Tri rapide randomisé (où le pivot est choisit aléatoirement uniformément) est en $O(n^2)$ mais en espérence elle est en $O(n \log n)$

Démonstation: On note $s_1 < s_2 < ... < s_n$ les éléments de S, on peut supposer les inégalités strictes, sinon on trie selon l'ordre lexicograpgique sur (valeur position)

Pour $1 \leq i < j \leq n$, on note X_{ij} le nombre de comparaisons faites entre s_i et s_j. On note X le nombre de comparaisons totales.

On cherche à calculer
$\mathbb{E}[X] = \mathbb{E}\left[\sum_{1 \leq i < j \leq n} X_{ij}\right] = \sum_{1 \leq i < j < n} \mathbb{E}\left[X_{ij}\right] = \sum_{1 \leq i < j \leq n} \left[1\mathbb{P}\left(X_{ij} = 1\right) + 0\mathbb{P}\left(X_{ij} = 0\right)\right].$

La dernière égalité est vraie car X_{ij} prend ses valeurs dans $\{0,1\}$, en effet si s_i et s_j sont comparés une fois, l'un des deux est pivot donc ne sera plus jamais comparé à quoi que ce soit

Pour calculer $\mathbb{P}\left(X_{ij} = 1\right)$, on considère le jeu suivant : $E_{ij} = \left\{s_i, ..., s_j\right\}$
On tire un pivot uniformément aléatoirement dans S, et :
1. Si le pivot est dans E_{ij}, et vaut s_i ou s_j, on arrête le jeu
2. Si le pivot est dans E_{ij}, et n'est ni s_i, ni s_j, on arrête le jeu
3. Si le pivot n'est pas dans E_{ij}, on recommence le tirage mais juste sur la moitié S' telle que $E_{ij} \subset S'$

Si on est dans 1, $X_{ij} = 1$
Si on est dans 0, $X_{ij} = 0$, car le pivot sépare les deux
Si on est dans trois, on ne sait pas encore car les deux peuvent encore être comparés

Ainsi, $\forall k \in \mathbb{N}, \mathbb{P}_{\text{le jeu a fait } k \text{ étapes}}\left(X_{ij} = 1\right) = \frac{2}{j-i+1}$
Ainsi, $\mathbb{P}\left(X_{ij}\right) = \sum_{k=1}^{+\infty} \mathbb{P}\left(X_{ij} = 1 \mid \text{le jeu a fait } k \text{ étapes}\right) \times \mathbb{P}(\text{jeu fait } k \text{ étapes})$
$= \frac{2}{j-i+1} \sum_{k \geq 1} \mathbb{P}(\text{le jeu fait } k \text{ étapes})$ où la somme fait $\mathbb{P}(\text{le jeu termine}) = 1$

On remplace dans $\mathbb{E}[X]$

$$
\begin{aligned}
\mathbb{E}[X] &= \sum_{1 \leq i < j \leq n} \mathbb{E}\left[X_{ij}\right] \\
&= \sum_{1 \leq i < j \leq n} \mathbb{P}\left(X_{ij} = 1\right) \\
&= \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{2}{j-i+1} \\
&= \sum_{i=1}^{n} \sum_{k=2}^{n-i+1} \frac{2}{k} \\
&\leq \sum_{i=1}^{n} \sum_{k=2}^{n} \frac{2}{k} \\
&= 2 \sum_{i=1}^{n} (H_n - 1) \\
&= 2n(H_n - 1) \\
&= O(n \log n)
\end{aligned}
$$

Avertissement

Complexité en espérance et complexité en moyene ne sont pas la même chose.

Complexité en espérance	Complexité en moyenne
Analyse d'un algorithme probabiliste	Analyse probabiliste d'un algorithme déterministe
On moyenne sur les choix probabilistes	On moyenne sur les entrées d'une taille donnée
On calcule le maximum sur les entrée de taille n du temps espéré sur l'entrée où le temps espéré sur l'enrée est la somme pour tout les choix possible du temps de calcul en faisant ce choix, pondérés par la probabilité du choix	On calcule $\frac{1}{\text{nombre d'entrées de taille n}} \times$ la somme du temps de calcul sur chaque entrée

II.3 Algorithmes de Monte Carlo

Définition

Un **algorithme de type Monte Carlo** est un algorithme probabiliste tel que son résultat dépend des choix probabilistes effectués, mais dont son temps de calcul pire cas est fixe

Exemple 5: La technique d'empruntes, par exemple :
But : Déterminer si un polynôme est identiquement nul
Algorithme :

```
Tirer uniformément aléatoirement x dans k
Si P(x) = 0, renvoyer vrai
sinon renvoyer faux
```

Définition

Si A est un algorithme Monte Carlo qui résout un problème de décision :

- Un **faux positif** est une exécution de A qui renvoie vrai sur une instance négative
- Un **faux négatif** est une exécution de A qui renvoie faux sur une instance positive

Un algorithme de Monte Carlo qui n'as pas de faux positifs ou pas de faux negatifs est dit **à erreur unilatérale**
S'il a des faux négatifs et des faux positifs, il est dit **à erreur bilatérale**

Exemple 6: L'algorithme précédent n'as pas de faux négatif car il ne peut pas renvoyer faux si P est effectivement nul, il est donc à erreur unilatérale.

Proposition 9: Si A est un algorithme Monte Carlo pour un problème sans faux négatifs et dont la probabilité de faux positifs est inférieur à p.
Alors pour tout $k \in \mathbb{N}^*$, k itérations indépendantes de A est également un algorithme de Monte Carlo sans faux négatifs avec une probabilités de faux positifs inférieure à p^k

Démonstation: Cet algorithme n'as pas de faux négatif sinon A en aurait De plus, la probabilité de faux positifs est la proba que A réponde incorrectement k fois sur une instance négative ce qui arrive avec une probabilité inférieure à p^k par indépendance des itération.

Ainsi, si on a un algorithme de Monte Carlo à erreur unilatérale alors on peut créer un algorithme de Monte Carlo à erreur unilatérale avec une probabilité de bonne réponse aussi grande que l'on veut, ce procédé s'appelle **l'amplification** de l'algorithme

II.4 Liens entre algorithmes Las Vegas et Monte Carlo

Idée

Sous certaines conditions, on peut convertir un algorithme d'un type en l'autre selon ce qu'on préfère

II.4.a Las Vegas vers Monte Carlo

Dans la résolutions des problèmes NP-Complet, il est fréquent de construire des algorithmes Las Vegas du type :

```
Tant qu'aucune solution n'as été trouvée
    Construire un candidat à être solution aléatoirement
    S'il est solution, le renvoyer
```

On peut les transformer en algorithme Monte Carlo en bornant le nombre de candidats qu'on essaie et en renvoyant le dernier candidat testé si on a atteint la borne

II.4.b De Monte Carlo à Las Vegas

Proposition 10: Si A est un algorithme Monte Carlo résolvant un problème et V est un vérificateur, c'est à dire un algorithme capable de vérifier si une sortie y est bien solution pour instance x de $\mathcal{P}$
Alors on peut construire un algorithme B de type Las Vegas qui résout $\mathcal{P}$
De plus, si le temps d'exécution de A sur une entrée de taille n est $f(n)$, si le temps d'exécution de V sur une entrée de taille n est $g(n)$ et si la probabilité d'erreur de A est p, alors l'espérance du temps de calcul de B est $\frac{1}{1-p} \times (f+g)$

Démonstation: B consiste à faire les instructions suivantes sur x :

```
Calculer A(x)
Si V indique que A(x) est solution pour x : renvoyer A(x)
Sinon : recommencer
```

Comebien d'opération fait-on en espérance sur une enttrée de taille n ?

Chaque itération coût $f(n)+g(n)$, on en fait en espérance $\mathbb{E}[X]$ avec X la variable aléatoire qui compte le nombre d'itérations.
$\mathbb{E}[X] = \sum_{k\geq 1} k\mathbb{P}(X=k) = \sum_{k\geq 1} kp^{k-1}(1-p)$
En effet, $X=k$ quand A s'est trompé $k-1$ fois puis on a vu juste à l'itération d'après, et par indépendance des variables
Ainsi, $\mathbb{E}[X] = (1-p)\sum_{k\geq 1} kp^{k-1} = \frac{1-p}{(1-p)^2}$, car on reconnaît la dérivée d'une série géométrique.

II.5 Exemple : algorithme de Freivalds

Le problème qu'on cherche à résoudre est :

$$\begin{cases} \underline{\text{Entrée :}}\ (A,B,C) \in M_n(\mathbb{Q}) \\ \underline{\text{Question :}}\ AB = C \quad ? \end{cases}$$

Algorithme de Freivalds :

```
Freivalds(A,B,C):
  Tirer uniformément aléatoirement X dans Mn,1({0,1})
  Si A(BX) = CX renvoyer vrai
  Sinon, renvoyer faux
```

Un algorithme déterministe pour résoudre ce problème est en $O(n^2)$.
L'algorithme de Freivalds est un algorithme Monte Carlo sans faux négatif. Sa probabilité de faux positifs est inférieur à $\frac{1}{2}$.

Démonstation: Soit A, B, C tels que $AB \neq C$,
déterminons $\mathbb{P}(\text{Freivalds}(A, B, C) = \text{oui})$

On note $D = AB - C \neq 0$
Ainsi, $\exists (i, j), d_{ij} \neq 0$

Observons la ligne i de DX, $l_i = d_{ij}x_j + r$ avec r le reste des termes.

$$\begin{aligned}
\mathbb{P}(l_i = 0) &= \mathbb{P}(l_i = 0 \wedge r = 0) + \mathbb{P}(l_i = 0 \wedge r \neq 0) \\
&= \mathbb{P}(l_i = 0 \mid r = 0)\mathbb{P}(r = 0) + \mathbb{P}(l_i = 0 \mid r \neq 0)\mathbb{P}(r \neq 0) \\
&\leq \mathbb{P}\big(x_j = 0\big)\mathbb{P}(r = 0) + \mathbb{P}\big(x_j \neq 0\big)\mathbb{P}(r \neq 0) \\
&\leq \frac{1}{2}\mathbb{P}(r = 0) + \frac{1}{2}(1 - \mathbb{P}(r = 0)) \\
&\leq \frac{1}{2}
\end{aligned}$$

De plus, $\mathbb{P}(\text{Freivalds}(A, B, C) = \text{oui}) = \mathbb{P}(DX = 0) \leq \mathbb{P}(l_i = 0) \leq \frac{1}{2}$